

수행내용 보고서
(2차 보고)

과제명	요트 다이스 보드게임 구현		
발표자	김승호	발표일자	2024.06.03
연구참여자 (담당분야)	팀장 : 이현영 (각 기능에 맞는 사용자 정의함수 작성, 과제총괄 및 보고서 작성) 팀원 : 강현욱 (각 기능에 맞는 사용자 정의함수 작성, 게임의 주요 알고리즘 구현) 팀원 : 김승호 (각 기능에 맞는 사용자 정의함수 작성, 코드 최적화) 팀원 : 이운상 (각 기능에 맞는 사용자 정의함수 작성, 테스트 및 디버깅)		
수업명/조번호	프로그래밍설계/5분대기조		

1. 연구일정 점검

- 2024년 4월 ~ 2024년 6월

수행 내용		일정					달성 실적
		1,2	3,4	5,6	7,8	9,10	
목표와 기준 설정	- 설계목표 설정						완료
	- 시장조사, 문헌조사						
	- 소프트웨어 요구사항 파악						완료
합성	- 기능 블록 구성						완료
	- 기능별 구현방법 결정						
	- 적용할 이론 및 기술						
	- UI 설계						
분석	- 세부 기능 블록도						완료
	- 목표달성가능성 확인						
	- 순서도 작성						
제작	- 프로그램 코딩						진행중
	- UI 구현						
시험/평가	- 시험 및 검증						
	- Trouble shooting						
	- 재설계						
결과	- 결과보고 및 시연						

2. 연구수행 내용

1) 규칙

```
#include <stdio.h>

void file_PrintRule()
{
    RuleFile("rule1.txt", "규칙1페이지 내용 ");
    RuleFile("rule2.txt", "규칙2페이지 내용");

    int choice;

    while (1)
    {
        printf("메뉴를 선택하세요:\n");
        printf("1. 규칙 1 페이지 출력\n");
        printf("2. 규칙 2 페이지 출력\n");
        printf("3. 종료\n");
        printf("선택: ");
        scanf("%d", &choice);

        switch (choice)
```

```

    {
        case 1:
            FileContent("rule1.txt");
            break;
        case 2:
            FileContent("rule2.txt");
            break;
        case 3:
            printf("프로그램을 종료합니다.\n");
            return;
        default:
            printf("잘못된 선택입니다. 다시 선택하세요.\n");
            break;
    }
}

int main()
{
    file_PrintRule();
    return 0;
}

```

2) 플레이어 정보

```

#include <stdio.h>
#include <string.h>

#define MAX_PLAYERS 4
#define NICKNAME_LENGTH 11

void PLAYER() {
    int playerCount;
    char nickname[MAX_PLAYERS][NICKNAME_LENGTH];
    int i;

    while (1) {
        printf("플레이어 수를 입력하세요 (최대 4명): ");
        scanf("%d", &playerCount);
        if (!(2 <= playerCount && playerCount <= 4)) {
            printf("최소 2명, 최대 4명까지 가능합니다. 다시 입력해주세요.\n");
        }
        else {
            break;
        }
    }

    for (i = 0; i < playerCount; i++) {
        printf("플레이어 %d의 닉네임을 입력하세요(영문 10자): ", i + 1);
        scanf("%s", nickname[i]);
    }
}

```

```

    for (i = playerCount; i < MAX_PLAYERS; i++) {
        strcpy(nickname[i], "-");
    }

    for (i = 0; i < MAX_PLAYERS; i++) {
        printf("플레이어 %d의 닉네임: %s\n", i + 1, nickname[i]);
    }
}

```

3) 점수판

```

#include "scoreboard.h"

void gotoxy(int x, int y) {
    COORD Pos;
    Pos.X = x;
    Pos.Y = y;
    SetConsoleCursorPosition(GetStdHandle(STD_OUTPUT_HANDLE), Pos);
}

char nickname[4][9] = { //닉네임 테스트
    {"현영"},
    {"승호"},
    {"현욱"},
    {"윤상"}
};

int p1score[12]; //플레이어별 점수배열
int p2score[12];
int p3score[12];
int p4score[12];
int Expectedscore[12];

int midTotal1, midTotal2, midTotal3, midTotal4;
int Total1, Total2, Total3, Total4;

void initSB() { // 점수와 점수판 초기화
    system("cls");
    for (int i = 0; i < 13; i++) {
        p1score[i] = 0;
        p2score[i] = 0;
        p3score[i] = 0;
        p4score[i] = 0;
    }
}

void printSB() { //점수판
    const char* category[] = { "Ones","Twos","Threes","Fours","Fives","Sixes","Choice", "Four of a Kind","Full House","Small Straight","Large Straight","Yacht", };

```



```

    }
}

void print_dice(int dice[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", dice[i]);
    }
    printf("\n");
}

char ask_reroll() {
    char answer;
    while (1) {
        printf("주사위를 더 굴리시겠습니까? (y/n): ");
        if (scanf_s(" %c", &answer) == 1 && (answer == 'y' || answer == 'n')) {
            break;
        }
        printf("잘못된 입력입니다. 'y' 또는 'n'을 입력하세요.\n");
        while (getchar() != '\n'); // 잘못된 입력 버퍼 비우기
    }
    return answer;
}

int get_reroll_count() {
    int reroll;
    while (1) {
        printf("몇 개의 주사위를 다시 굴리시겠습니까? (1-5): ");
        if (scanf_s("%d", &reroll) == 1 && reroll >= 1 && reroll <= 5) {
            break;
        }
        printf("잘못된 입력입니다. 1에서 5 사이의 숫자를 입력하세요.\n");
        while (getchar() != '\n'); // 잘못된 입력 버퍼 비우기
    }
    return reroll;
}

void get_reroll_indices(int reroll_indices[], int reroll) {
    while (1) {
        printf("다시 굴릴 주사위의 번호를 선택하세요 (1-5, 공백으로 구분): ");
        int dices_entered = 0;
        int invalid_input = 0;
        for (int i = 0; i < reroll; i++) {
            int index;
            if (scanf_s("%d", &index) != 1 || index < 1 || index > 5 || reroll_indices[index - 1]) {
                if (index < 1 || index > 5) {
                    printf("잘못된 번호입니다. 1에서 5 사이의 숫자를 입력하세요.\n");
                }
                else if (reroll_indices[index - 1]) {
                    printf("이미 선택된 번호입니다. 다른 번호를 선택하세요.\n");
                }
            }
            invalid_input = 1;
        }
    }
}

```

```

        while (getchar() != '\n'); // 잘못된 입력 버퍼 비우기
        break;
    }
    reroll_indice[index - 1] = 1;
    dices_entered++;
}
if (invalid_input && dices_entered == reroll) {
    break;
}
else {
    printf("잘못된 입력입니다. %d개의 유효한 번호를 입력하세요.\n", reroll);
    while (getchar() != '\n');
    for (int i = 0; i < 5; i++) {
        reroll_indice[i] = 0; // 재입력을 위해 배열 초기화
    }
}
}
}

void play_game() {
    int dice[5];
    int b = 0;

    printf("주사위를 한 번 굴립니다.\n");
    roll_dice(dice, 5);
    printf("주사위 조합: ");
    print_dice(dice, 5);

    while (1) {
        char answer = ask_reroll();
        if (answer == 'n') {
            break;
        }
        if (answer == 'y' && b < 3) {
            b++;
            int reroll = get_reroll_count();
            int reroll_indice[5] = { 0 };
            get_reroll_indices(reroll_indice, reroll);
            for (int i = 0; i < 5; i++) {
                if (reroll_indice[i]) {
                    dice[i] = rand() % 6 + 1;
                }
            }
            printf("주사위 조합: ");
            print_dice(dice, 5);
        }
        if (b >= 3) {
            break;
        }
    }
    printf("턴을 종료합니다.\n");
}

```

```
}
```

5) 주사위 정렬

```
#include <stdio.h>
int selection_sort(int dice[])
{
    int n = 5;
    int i, j, min, temp;
    int k;

    for (i = 0; i < n - 1; i++)
    {
        min = i;
        for (j = i + 1; j < n; j++)
        {
            if (dice[j] < dice[min])
                min = j;
        }
        temp = dice[min];
        dice[min] = dice[i];
        dice[i] = temp;
    }
    for (k = 0; k < n; k++)
        printf("%d ", dice[k]);
    printf("\n");
}

int main() {
    int arr[5] = { 5, 5, 2, 1, 1 }; // 예시
    printf("주사위 정렬: ");
    selection_sort(arr);
    return 0;
}
```

6) 주사위 모양 출력

```
#include "printdice.h"

// 주사위들의 모양 정하기
char dicepattern1[5][30] = {
    {" ┌──────────┐ "},
    {" │          │ "},
    {" │  ●        │ "},
    {" │          │ "},
    {" └──────────┘ "}
};

char dicepattern2[5][30] = {
    {" ┌──────────┐ "},
    {" │ ●          │ "},
    {" │          │ "},
    {" │          │ "},
    {" └──────────┘ "}
```



```

    {" |      ● | "},
    {" └────────┘ "}
};

char dicepattern3[5][30] = {
    {" └────────┘ "},
    {" | ●      | "},
    {" |      ● | "},
    {" |      ● | "},
    {" └────────┘ "}
};

char dicepattern4[5][30] = {
    {" └────────┘ "},
    {" | ●      ● | "},
    {" |      | "},
    {" | ●      ● | "},
    {" └────────┘ "}
};

char dicepattern5[5][30] = {
    {" └────────┘ "},
    {" | ●      ● | "},
    {" |      ● | "},
    {" | ●      ● | "},
    {" └────────┘ "}
};

char dicepattern6[5][30] = {
    {" └────────┘ "},
    {" | ●      ● | "},
    {" | ●      ● | "},
    {" | ●      ● | "},
    {" └────────┘ "}
};

```

```

void printDice(int dice[5]) { // 주사위 출력 함수
    for (int i = 0; i < 5; i++) {
        int j;
        switch (dice[i]) {
            case 1:
                for (j = 0; j < 5; j++) {
                    gotoxy(i * 11 + 2, j); //해당 x좌표 위치에 줄바꿈 해가면서 출력
                    printf("%s", dicepattern1[j]);
                }
                break;
            case 2:
                for (j = 0; j < 5; j++) {
                    gotoxy(i * 11 + 2, j);
                    printf("%s", dicepattern2[j]);
                }
            }
        }
    }
}

```

```

        break;
    case 3:
        for (j = 0; j < 5; j++) {
            gotoxy(i * 11 + 2, j);
            printf("%s", dicepattern3[j]);
        }
        break;
    case 4:
        for (j = 0; j < 5; j++) {
            gotoxy(i * 11 + 2, j);
            printf("%s", dicepattern4[j]);
        }
        break;
    case 5:
        for (j = 0; j < 5; j++) {
            gotoxy(i * 11 + 2, j);
            printf("%s", dicepattern5[j]);
        }
        break;
    case 6:
        for (j = 0; j < 5; j++) {
            gotoxy(i * 11 + 2, j);
            printf("%s", dicepattern6[j]);
        }
        break;
    }
    // Sleep(500); 주사위 하나씩 나오는 모습 연출
}
}
}

```

7) 점수 계산

```

#include <stdio.h>

int* score_cal(int 주사위[]) {
    static int score[12] = { 0 }; // 점수 족보 배열을 정적으로 선언
    int count[6] = { 0 };         // 주사위 눈의 개수를 세기 위한 배열
    int temp[3] = { 0 };

    for (int i = 0; i < 5; i++) { // 1~6 페어 (점수 족보 배열의 0번부터 5번까지)
        score[주사위[i] - 1] += 주사위[i];
        score[6] += 주사위[i]; // 초이스 (점수 족보 배열의 6번)
        count[주사위[i] - 1]++;
    }

    for (int i = 0; i < 6; i++) {
        if (count[i] == 2)
            temp[0] = 1; // 폴하우스(주사위 같은 수 2개 확인)
        else if (count[i] == 3)
            temp[1] = 2; // 폴하우스(주사위 같은 수 3개 확인)
        if (count[i] != 0) //temp2 =>연속된 숫자인지를 확인하는 임시
    }
}

```

```

        temp[2] += 1;
    else if (count[i] == 0) //연속되는 수가 없으면 0으로 초기화
        temp[2] = 0;

    if (count[i] >= 4) { // 포카드 (점수 족보 배열의 7번)
        score[7] = score[6];
    }
    if (temp[2] == 4) { // 스물 스트레이트 (점수 족보 배열의 9번)
        score[9] = 15;
    }
    if (temp[2] == 5) { // 라지 스트레이트 (점수 족보 배열의 10번)
        score[10] = 30;
    }
    if (temp[0] == 1 && temp[1] == 2) { // 풀하우스 (점수 족보 배열의 8번)
        score[8] = score[6];
    }
    if (count[i] == 5) { // 요트 (점수 족보 배열의 11번)
        score[11] = 50;
    }
}
return score;
}

```

8) 점수추천 및 선택

```

#include <stdio.h>

int recommendScore(int expectedScores[]) {
    int realScore, category;
    int scoreType = 1;

    while (1) {
        int expectedScore = expectedScores[scoreType - 1];
        if (expectedScore != 0) {
            if (scoreType == 5) {
                for (int i=0; i<6; i++) {
                    gotoxy(61, 8 + 2 * i);
                    // 점수 출력
                }
                gotoxy(61, 8 + 2 * 6);
                // 중간 점수 출력
            }

            if (scoreType == 11) {
                for (int i=6; i<12; i++) {
                    gotoxy(61, 22 + 2 * i);
                    // 점수 출력
                }
                gotoxy(61, 8 + 2 * 12);
                // 최종 점수 출력
            }
        }
    }
}

```

```

    } else {
        printf("-Wn");
    }

    if (scoreType < 12) {
        scoreType++;
    } else {
        printf("원하는 카테고리를 입력하시오(1 ~ 12) : ");
        scanf_s("%d", &category);
        realScore = expectedScores[category - 1];
        break;
    }
}

return realScore;
}

```

9) 승패 여부

```

#include <stdio.h>

// 배열로 플레이어 점수 전달받음
int determineWinner(int scores[]) {
    int maxScore = scores[0];
    int winner = 1;

    for (int i = 1; i < PLAYERS; i++) {
        if (scores[i] > maxScore) {
            maxScore = scores[i];
            winner = i + 1;
        }
    }

    return winner;
}

```

3. 연구계획 (향후 연구수행 예정인 내용 요약)

구분	수행내용 요약
팀전체	모듈화, 소스코드의 시험 및 평가

4. 문제점 및 애로사항 :

- 1) gotoxy함수 : 함수를 처음 써봐서 인터페이스를 만드는 과정 중 원하는 위치에 정확하게 출력하는 것이 어려웠다.
- 2) 주사위굴리기 모듈 : 해당 모듈은 사용자로부터 숫자를 입력받아야하는 경우가 많은데 잘못된 숫자를 입력받았을 때 오류가 나거나 무한루프에 빠지는 등 문제가 있었지만, getchar 함수를 이용해 입력버퍼를 비우는 것으로 해결하였다.

5. 지도교수 코멘트 내용 :

회 의 록				
회의명	최종 계획 논의			
일 시	2024.05.24		장 소	온라인
의 제	모듈 별 소스코드 피드백, 향후 계획 논의			
회의 내용	1. 최종 모듈화를 위한 계획 논의 2. 역할 분담하였던 각각의 소스코드 피드백			
결의 사항	1. 헤더파일이 너무 많아지지 않도록 묶을 수 있는 것들에 대해 논의하였고, 모듈화에 관한 역할 분담을 하였다 2. 저번에 각자 역할 분담하여 만들어 온 소스코드를 피드백하였고, 묶을 수 있는 소스코드끼리 모듈화를 위한 헤더파일을 만들기로 하였다.			
이견 사항	없음			
참석 현황	대상	성명	학번	서 명(자필)
	참석자	강현욱	20210885	
		김승호	20210901	
		이윤상	20210947	
		이현영	20200936	
	불참자			
불참 사유				
작성자	이현영	작성일	2024.05.24	

지도교수 상담일지

과제명			
일지 NO		학년도/학기	2021 학년 / 1 학기
소속	정보통신공학과 2학년		
지도교수 확인	2021년	월	일 지도교수: (인)
상담 안건			
상담 내용			
코멘트에 대한 조치사항			
상담 참여자	_____ (인)	_____ (인)	
	_____ (인)	_____ (인)	